

especially in rapid succession, is worth a look. A spike in 500 errors on an endpoint that's been quiet for months is interesting.

Tools like Datadog, Grafana, or Elastic Stack can help you set up these kinds of alerts. The key is to alert on patterns that are actually unusual, not just on raw thresholds, or you'll drown in noise and stop paying attention.

Secrets, tokens, and the mess people make of them

I covered this in the zero-trust article (*OSFY*, June 2026), but it's worth revisiting in the API-specific context because the patterns are slightly different.

API keys are still everywhere, and that's mostly okay if you treat them like passwords. Rotate them. Don't commit them to git. Use a secrets manager. Give each integration its own key so you can revoke them individually. Set expiry dates and enforce them.

For internal service-to-service calls, prefer short-lived tokens issued by an identity service over long-lived API keys. The pattern is similar to user authentication, just with services as the subject. SPIFFE-style workload identities are great for this if you're at the scale where managing tokens by hand becomes a chore.

For partner APIs and public APIs, OAuth 2.0 with proper scopes is your friend. Define scopes that mean something. *read:invoices* and *write:invoices* are different things and should be granted separately. Don't fall into the trap of giving every integration full access just because it's easier.

And please, please, do not put API keys in mobile apps and call them secret. Anyone with a few minutes and the right tools can pull every string out of your APK or IPA. Mobile apps should authenticate through a user session, not through a shared secret embedded in the binary.

When AI agents are calling your API

This is the new wrinkle and it's worth thinking about specifically. AI agents acting on behalf of users are starting to

show up as legitimate callers. They're going to make requests that look weird. They're going to retry things. They're going to hallucinate parameters and send you nonsense. They're also going to be targeted by attackers trying to manipulate them into doing things they shouldn't.

A few things help. First, treat agent traffic as a distinct caller category if you can. Issue tokens specifically for agent use, with scopes that reflect what an agent should be allowed to do on the user's behalf. Don't just give them the user's full session.

Second, watch for unusual patterns. An agent that suddenly starts trying to access a different user's data is either compromised or being manipulated. Either way, that's worth alerting on.

Third, think hard about destructive actions. If an agent can delete records, transfer money, or send messages to other people, those actions should require additional confirmation or be subject to stricter rate limits. The cost of an agent making a mistake on a read endpoint is small. The cost of an agent being tricked into wiring money somewhere is not.

Prompt injection, where an attacker tries to manipulate an AI agent through cleverly crafted input, is a whole topic of its own. The short version is that your API can't assume the agent is operating with clean inputs. Treat agent requests as semi-trusted, not fully trusted, and design accordingly.

Versioning, deprecation, and the long tail of old endpoints

Here's a quiet problem that gets very loud at the worst times. You released version 1 of your API four years ago. You released version 2 two years ago. You released version 3 last month. You documented version 3. You forgot about version 1 entirely. And it's still out there, still serving traffic, still using the authentication patterns you've since abandoned.

This is how a lot of breaches happen. Not through the shiny new endpoint. Through the forgotten old one.

Build deprecation into your API design from day one. Mark old versions clearly. Communicate end-of-life dates aggressively. Turn off endpoints when their time is up, even if it means breaking the one customer who never migrated. Especially if it means breaking the one customer who never migrated.

Audit your API surface regularly. Pull a list of every endpoint that's receiving traffic. Compare it to what's documented. Anything on the list that isn't in the docs is a candidate for removal. Anything in the docs that isn't on the list is a candidate for being formally deprecated.

What you can do this week, this month, this quarter

This week, pick your three highest traffic endpoints. Look at the authorisation logic. Make sure each one checks not just authentication but also whether the caller is allowed to do what they're asking. If any of them just check "is this user logged in" and then return the requested object regardless of ownership, fix that first.

This month, audit your authentication. Are tokens actually being verified properly? Are signatures being checked? Are expiry times reasonable? Is there a way to revoke a token if it leaks? If the answer to any of these is "I'm not sure," go find out.

Add rate limiting to your login endpoint, your password reset endpoint, and any endpoint that returns sensitive data or sends emails. Five per fifteen minutes is a reasonable starting point for human-facing endpoints.

This quarter, set up structured logging if you don't have it. Ship the logs somewhere central. Set up alerts on the patterns that matter. Review your dependency tree for known vulnerabilities, ideally with an automated tool that runs on every build. Tools like Snyk, Dependabot, or npm audit do most of the work for you.

Then keep going. Security isn't a project you finish. It's a habit that gets a bit stronger every sprint.

■■ Continued to page...76